



Fase 1b — De un Repositorio de Código a un Skill de Referencia (API/Código)

> Documento de estudio técnico | Complementa la Fase 1 (libro) y la Fase 2 (resolución)

Ficha Técnica del Documento

📅 Fecha:	Septiembre 2026	🏗️ Arquitectura:	3 Capas Deterministas
🔗 Entrada:	Repo GitHub o dir local	🔗 Perfil de síntesis:	referencia_codigo
📦 Modelo (default):	Tier deepseek	🛡️ Validación:	Prob. de Sistema (0 errores E2E)

Índice

1. 🎯 Qué Resuelve la Fase 1b	1
1.1. Diferencia frente a la Fase 1 (libro)	2
2. 🏗️ Arquitectura de 3 Capas (obligatoria)	2
3. ⚙️ El Orquestador <code>flujo_repo_a_skill.py</code>	2
4. 📦 El Perfil <code>referencia_codigo</code> en <code>synthesize_skill.py</code>	4
4.1. Prompts de destilación por chunk	4
4.2. Esquema de references y normas de contenido	4
5. 🛡️ La Validación en el Perfil <code>referencia_codigo</code>	4
6. 💰 Costo y Créditos	5
7. ➤ Comandos de Ejemplo	5
8. ⚠️ Edge Cases Clave (ver la directiva)	6

1 🎯 Qué Resuelve la Fase 1b

Un LLM que “recuerda” una librería (`~/config/opencode/skills/<nombre>/README.md`) tiende a alucinar sus APIs. La **Fase 1b** transforma un **repositorio (GitHub o local)** en un **skill estructurado de referencia de código/API**:

```
1 [GitHub URL | dir local] --> extraer_repo_github.py --> texto_completo.
   txt
2
3                                     |
4                                     [enrutador.py (
5                                     contexto_masivo)]
6                                     |
7                                     sintetizar_skill.py --perfil referencia_codigo
8                                     (destila chunk a chunk --> corpus; ensambla 6
9                                     references)
10                                    |
11                                    validar_skill_formulas.py --perfil referencia_codigo (
12                                    gate)
13                                    |
14                                    generar_latex_skill.py --> reporte PDF | instalar_skill.py -->
15                                    skills
```

1.1 Diferencia frente a la Fase 1 (libro)

Eje	Fase 1 (libro)	Fase 1b (repo)
Fuente	PDF de texto	Repositorio (URL GitHub o directorio local)
Extracción	extraer_libro_pdf.py	extraer_repo_github.py (clona, filtra, concatena)
Perfil de síntesis	libro (default)	referencia_codigo
Esquema de referencias	formulas.md, metodologias.md, limites_aplicabilidad.md, tablas.md...	api.md, patrones.md, ejemplos.md, configuracion.md, prerequisites.md, glosario.md
Código en bloques	SymPy autocontenido (se ejecuta en sandbox)	Snippets/ejemplos reales del repo (from <paquete> import ...)
Validación	Sintaxis + imports seguros (solo math/sympy/fractions) + ejecución en sandbox	Sintaxis (observación) + bloqueo solo de módulos/funciones de Sistema; NO ejecuta en sandbox

Tabla 1: Comparativa entre la Fase 1 (destilación de libro) y la Fase 1b (destilación de repositorio).

2 Arquitectura de 3 Capas (obligatoria)

3 El Orquestador flujo_repo_a_skill.py

Pasos (total depende de flags):

Capa	Archivo	Rol
Directiva	directives/repo_a_skill.yaml	SDP: 8 pasos, 8 edge cases, retry budget
Orquestación	flujo_repo_a_skill.py	Entrevista, orquesta scripts, gate de instalación
Ejecución	execution/extraer_repo_github.py execution/sintetizar_skill.py (perfil referencia_codigo) execution/validar_skill_formulas.py (perfil referencia_codigo) execution/generar_latex_skill.py execution/instalar_skill.py	Trabajo determinista

Tabla 2: Matriz de responsabilidades de la arquitectura de 3 capas en la Fase 1b.

1. **Entrevista ligera:** fuente, tema, nombre (snake_case, default derivado del repo), idioma, filtros de archivos y confirmación de instalación. Se persiste en `.tmp/entrevista_skill_<nombre>.json`.
2. **Extracción** — `execution/extraer_repo_github.py` (determinista, 0 créditos):
 - URL GitHub → `git clone -depth 1 a .tmp/repo_clone_<owner>-<repo>/`; ruta local → usa el dir tal cual.
 - Filtra por extensión (docs + código fuente) saltando `.git/node_modules/vendor/build/venv/__pycache__`.
 - `-incluir/-excluir` globs opcionales, `-max-archivos/-max-bytes`.
 - Concatena a `.tmp/repo_<nombre>_texto/texto_completo.txt` con cabeceras `=== ARCHIVO: <ruta> ===` y genera índice.json con árbol de estructura.
 - E2E real: `pallets/itsdangerous` → 32 archivos, ~22.5K tokens.
3. **Enrutamiento determinista** — `execution/enrutador.py` --taskcontexto_masivo: mide tokens y decide tier. Para un repo grande → **deepseek** (deepseek/deepseek-v4-pro).
4. **Síntesis (créditos)** — `execution/sintetizar_skill.py` --perfilreferencia_codigo → 7 archivos (SKILL.md + 6 references de API). Prompts adaptados: NO fuerzan SymPy.
5. **Validación (0 créditos)** — `execution/validar_skill_formulas.py` --perfilreferencia_codigo. Solo bloquea riesgos de Sistema; los snippets de API (firmas incompletas, imports del paquete) se marcan ok/observacion, nunca errores.
6. **Reporte LaTeX (0 créditos)** — `execution/generar_latex_skill.py` → PDF en docs/SKILL/<nombre>/.
7. **Revisión humana** del skill.
8. **Instalación (si no es -dry-run)** — `execution/instalar_skill.py` → copia a `~/ .config/opencvode/skills/<nombre>/`. Recuerda reiniciar opencode.

4 El Perfil referencia_codigo en sintetizar_skill.py

-perfil (default libro) cambia dos cosas:

4.1 Prompts de destilación por chunk

_prompts_chunk_codigo: en vez de “fórmulas, metodologías, tablas”, pide “funciones y clases públicas con firmas, patrones de uso, configuración relevante, ejemplos de código que el lector pueda copiar”.

4.2 Esquema de references y normas de contenido

_normas_contenido_codigo:

Archivo	Contenido
api.md	Funciones/clases públicas con firma completa (parámetros, retorno) y ejemplo de llamada
patrones.md	Cómo se combinan funciones/clases para resolver problemas típicos
ejemplos.md	Código REAL del repo (no inventado), marcado por archivo de origen
configuracion.md	Opciones, variables de entorno, archivos de config
prerrequisitos.md	Dependencias, versiones, cómo instalar
glosario.md	Términos técnicos del proyecto

Tabla 3: Contenido de las 6 referencias en el perfil referencia_codigo.

Además: el **frontmatter** usa el mismo saneo determinista (_normalizar_frontmatter) y _find_balanced_json aplica igual (robusto a llaves anidadas).

5 La Validación en el Perfil referencia_codigo

El validador clásico (validar_skill_formulas.py perfil libro) exige que cada bloque python sea un programa SymPy autocontenido (solo math/sympy/fractions) y lo ejecuta en sandbox. Eso es **incorrecto para un skill de API**: los ejemplos legítimamente hacen from <paquete> import ..., y el paquete no está instalado en el sandbox.

Con -perfil referencia_codigo:

1. **Sintaxis** — ast.parse de cada bloque. Si falla (firma/documentación incompleta), se marca observacion (NO bloquea): son snippets ilustrativos, no programas.
2. **Seguridad** — _escaneo_peligro_sistema: solo bloquea módulos/funciones de Sistema con riesgo real

(os, sys, subprocess, socket, ctypes, open, eval, exec, __import__, ...). Imports de librería/proyecto (itsdangerous, hashlib) son legítimos. Un bloque con riesgo → estado peligro (ÚNICO estado que cuenta como error y hace exit 3).

3. **NO se ejecuta en sandbox** — el código depende del paquete del repo.

Punto crítico de diseño

Decisión de diseño verificada en E2E: un sandbox que ejecutara los ejemplos de una API convertiría 74 bloques legítimos (imports del paquete, firmas parciales) en 74 falsos positivos. El perfil adaptativo reduce eso a **0 errores** conservando la protección real contra código peligroso de Sistema.

6 Costo y Créditos

- **0 créditos:** extracción, enrutador (decisión local), validación, LaTeX, tests.
- **Créditos:** síntesis (openrouter_chat, default deepseek). Repo pequeño: ~22K tokens de entrada ≈ centavos de dólar. Repo grande: avisar antes, acotar con -incluir/-excluir.
- Verificación de saldo puntual: `python3 execution/monitor_saldo_openrouter.py`.

7 Comandos de Ejemplo

Ejemplos de invocación

```

1 # GitHub publico, disponible para resolver
2 python3 flujo_repo_a_skill.py --repo https://github.com/pallets/
   itsdangerous \
3   --tema "Firma y autenticacion de datos" --nombre
   pallets_itsdangerous
4
5 # Directorio local acotando a codigo Python
6 python3 flujo_repo_a_skill.py --repo /ruta/al/repo --nombre
   midu_skill \
7   --incluir "*.py" "*.md" --excluir "tests/*" --dry-run
8
9 # Solo extraer (0 creditos, determinista)
10 python3 execution/extraer_repo_github.py --fuente https://github.
   com/psf/requests \
11   --salida .tmp/repo_requests_texto

```

8 Edge Cases Clave (ver la directiva)

Caso	Recuperación
Repo privado / 404	git clone falla; ofrecer ruta local o GITHUB_TOKEN
Repo sin código/docs tras filtrar	Código 2; sugerir <code>-include</code>
Repo enorme / costo alto	Avisar tokens medidos; acotar con globs o límites
Snippets no autocontenidos	observacion (no bloquea instalación) en perfil código
Bloque con riesgo de Sistema	peligro → exit 3; con <code>-validar-estricto</code> aborta instalación

Tabla 4: Catálogo de casos límite y protocolo de recuperación (directiva `repo_a_skill.yaml`).

Recuerda

El enfoque neuro-simbólico separa el razonamiento del LLM (probabilístico) de la decisión y la validación (deterministas). En la Fase 1b esto se concreta en: extracción determinista del repo, enrutamiento con umbrales en código, validación adaptativa por perfil y coste controlado por globs.